

PROMPT PER CLAUDE OPUS 4.6

Progetto: "Elden Ring Nightreign: The True Relic System"

OBIETTIVO GENERALE

Realizza un **sito web statico completo** (hostabile su GitHub Pages) per gestire il sistema di crafting delle reliquie del gioco **Elden Ring Nightreign**, secondo le regole custom descritte in questo prompt. Il sito deve essere interamente in **italiano**, con uno stile visivo **scuro, minimalista e moderno** (dark UI, font puliti, palette sobria con accenti dorati/ambra in tema fantasy).

Il sito è pensato per **3 giocatori** che possono accedere dal proprio dispositivo. Non esiste un backend: il sito lavora con un **file JSON** che funge da database, caricato e scaricato dall'utente.

ARCHITETTURA TECNICA

Stack

- **HTML5 + CSS3 + JavaScript vanilla** (nessun framework, nessun bundler), tutto in un unico file `index.html` per semplicità di deployment su GitHub Pages.
- Eccezione: i **file JSON di configurazione degli effetti** sono file separati che il sito legge dinamicamente (vedi sezione dedicata).

Gestione dati (pseudo-database)

- Il sito accetta in input e produce in output un file `reliquie.json` strutturato (schema definito sotto).
- Al caricamento, l'utente importa il file JSON. Il sito opera interamente in memoria. Al termine, l'utente esporta il file aggiornato.
- Ogni reliquia ha un campo `proprietario` (stringa: nome del giocatore).
- Il sito deve anche essere usabile **senza file** (partendo da zero), generando un nuovo

database vuoto.

File di configurazione effetti (separati, editabili dall'utente)

Il sito carica da percorso relativo i seguenti **3 file JSON**:

- data/effetti_normali_buff.json — pool di buff per reliquie normali
- data/effetti_notte_buff.json — pool di buff per reliquie della notte
- data/effetti_notte_debuff.json — pool di debuff per reliquie della notte

Schema di ciascun file:

```
[
  {
    "id": "atk_boost",
    "nome": "Aumento Attacco",
    "stepMax": 3,
    "descrizioneStep": ["Attacco +5%", "Attacco +10%", "Attacco +15%"]
  },
  ...
]
```

I file devono essere precompilati con almeno **10 effetti placeholder** ciascuno, con nomi verosimili. L'utente potrà modificarli liberamente senza toccare il codice del sito.

SCHEMA DEL DATABASE JSON (`reliquie.json`)

```
{
  "versione": "1.0",
  "giocatori": ["Giocatore1", "Giocatore2", "Giocatore3"],
  "reliquie": [
    {
      "id": "uuid-univoco",
      "nome": "Nome Reliquia",
      "proprietario": "Giocatore1",
      "grado": 2,
      "grandezza": "Piccola",
      "colore": "Rosso",
      "tipologia": "normale",
      "effetti": [
        { "id": "atk_boost", "step": 2 },
        { "id": "def_boost", "step": 1 }
      ]
    }
  ]
}
```

```

    ],
    "isFusa": false,
    "stadioCartena": 0,
    "storiaCrafting": []
  }
],
"registroFusioni": [
  {
    "id": "uuid-fusione",
    "data": "2026-02-17T20:00:00",
    "proprietarioRisultato": "Giocatore1",
    "materialiIds": ["id1", "id2"],
    "risultatoId": "id-nuova",
    "degradata": false,
    "testimoni": ["Giocatore2", "Giocatore3"]
  }
],
"registroToken": [
  {
    "id": "uuid",
    "data": "2026-02-17T20:00:00",
    "tipo": "cromatico",
    "giocatore": "Giocatore1",
    "descrizione": "Cambio colore reliquia X da Rosso a Blu"
  }
]
]
}

```

Regole sullo schema:

- I campi `grado`, `grandezza`, `colore`, `tipologia` sono enumerati (vedi sotto).
- `isFusa : true` se la reliquia è il risultato di una fusione.
- `stadioCartena` : intero ≥ 0 . Per reliquie non fuse = 0.
- `storiaCrafting` : array di ID fusioni che hanno coinvolto questa reliquia come materiale.
- `effetti` : array di oggetti `{id, step}` dove `step` è il livello attuale (1-based, $\leq \text{stepMax}$ dell'effetto).

Valori enumerati:

- `grado` : 1, 2, 3
- `grandezza` : "Piccola", "Media", "Grande"
- `colore` : "Rosso", "Blu", "Giallo", "Verde"

- `tipologia: "normale", "notte"`
-

STRUTTURA DEL SITO (SEZIONI/PAGINE)

Il sito è una **Single Page Application** con navigazione a schede (tab). Le sezioni sono:

1. **Home / Carica dati**
 2. **Inventario Reliquie**
 3. **Fusione - Reliquie Normali**
 4. **Fusione - Reliquie della Notte**
 5. **Token & Ricompense**
 6. **Registro Fusioni**
 7. **Impostazioni / Gestione Giocatori**
-

SEZIONE 1 — HOME / CARICA DATI

- Pulsante **"Carica file reliquie.json"** → input file che importa il database.
 - Pulsante **"Nuovo database vuoto"** → inizializza un database vuoto, chiede i nomi dei 3 giocatori.
 - Indicatore dello stato corrente: nome file caricato, numero reliquie, data ultima modifica.
 - Pulsante **"Esporta reliquie.json"** (visibile in tutta l'app, es. header) → scarica il file aggiornato.
 - Breve descrizione del sito e delle sue funzionalità.
-

SEZIONE 2 — INVENTARIO RELIQUIE

Vista lista

- Mostra tutte le reliquie del database in card o tabella.
- Filtri: per proprietario, per grado, per colore, per grandezza, per tipologia (normale/notte).

- Ordinamento: per grado, per proprietario, per data aggiunta.
- Badge visivi per colore (pallino colorato), grado (G1/G2/G3), tipologia (🌙 per notte).

Card reliquia

Ogni card mostra:

- Nome, proprietario, grado, grandezza, colore, tipologia
- Lista effetti con step attuale e step massimo (es. "Aumento Attacco — Step 2/3")
- Badge "Fusa" se `isFusa: true`, con indicazione dello stadio catena
- Pulsante "**Modifica**" → form inline per modificare nome e proprietario
- Pulsante "**Elimina**" → conferma prima di eliminare

Aggiunta manuale reliquia

- Pulsante "+ **Aggiungi Reliquia**" → form modale con tutti i campi.
- Gli effetti si aggiungono selezionando dall'elenco caricato dai JSON di configurazione.
- Al salvataggio, `isFusa: false`, `stadioCatena: 0`.

SEZIONE 3 & 4 — FUSIONE (NORMALI e NOTTE, sezioni separate ma stessa logica)

Questa è la sezione più complessa. Implementa un **wizard a step**.

STEP 1 — Selezione materiali

- L'utente seleziona **2 o più reliquie** dalla lista (filtrata per tipologia corrente).
- Validazione in tempo reale:
 - ☒ Tutte della stessa tipologia (normale o notte)
 - ☒ Tutte della stessa grandezza
 - ☒ Tutte di Grado ≥ 2
 - ☒ Nessuna reliquia di Grado 1 base
 - ☒ Mostrare errore bloccante se i vincoli non sono rispettati

- Mostra riepilogo materiali selezionati.
- Campo: seleziona il **proprietario della reliquia risultante** (dropdown con i 3 giocatori).

STEP 2 — Calcolo colore

- Calcola le probabilità ponderate del colore risultante.
 - Formula: per ogni colore, $\text{probabilità} = (\text{numero reliquie di quel colore}) / (\text{totale reliquie})$
- Mostra le probabilità in modo chiaro (es. barra o tabella: "Rosso 50%, Blu 25%, Verde 25%").
- Pulsante "**Estrai colore**" → simula il sorteggio ponderato e mostra il risultato con animazione.
- L'utente può ripetere l'estrazione (il regolamento prevede Wheel of Names, qui la simuliamo).

STEP 3 — Sorteggio effetti

Regole generali:

- Il pool di effetti disponibili è l'unione di tutti gli effetti delle reliquie materiali.
- La reliquia risultante ha tanti slot quanti il suo grado (che è lo stesso delle reliquie materiali, es. tutte G2 → risultato G2, 2 effetti).
- Si sorteggia un effetto alla volta dal pool residuo.

Gestione duplicati (Art. 7):

- Se un effetto viene estratto e la reliquia risultante ha già quell'effetto:
 - Se lo step attuale è < stepMax: incrementa lo step di 1 (upgrade). L'effetto rimane nella lista risultato aggiornato.
 - Se già a step massimo: scarta l'estrazione, sorteggia un altro effetto dal pool.
- **NORMA IMPERATIVA (Art. 8):** Ogni effetto può essere potenziato **al massimo una volta per singola fusione**, indipendentemente da quante copie esistano nel pool.

Per reliquie della Notte (sezione 4):

- I sorteggi di buff e debuff sono **separati**.
- Il numero di slot buff e debuff nella reliquia risultante dipende dagli effetti delle reliquie materiali (proporzione degli effetti nel pool).

- Implementa due ruote separate: una per i buff, una per i debuff.

UI dello step:

- Mostra il pool effetti attuale (con quante copie di ciascun effetto ci sono nel pool).
- Pulsante **"Estrai effetto"** → simula sorteggio, mostra risultato, aggiorna pool e lista effetti risultanti.
- Mostra la lista effetti della reliquia risultante aggiornandosi ad ogni estrazione.
- Indicazione se un effetto ha subito upgrade.

STEP 4 — Test di degradazione

Calcolo probabilità:

1. Rischio base: G2 = 10%, G3 = 20%
2. +10% per ogni reliquia **fusa** usata come materiale (campo `isFusa: true`)
3. Catena (Art. 11): per ogni reliquia materiale con `stadioCartena ≥ 4`, aggiungi $(\text{stadioCartena} - 3) * 10\%$
4. Somma tutti i contributi → probabilità totale degradazione

Calcolo stadio catena della reliquia risultante:

- `stadioCartena = max(stadioCartena di tutte le reliquie fuse usate come materiale) + 1`
- Se nessuna reliquia materiale è fusa: `stadioCartena = 0` (non è una catena)
- Se almeno una è fusa: `stadioCartena = max(...) + 1`

UI dello step:

- Mostra chiaramente il breakdown del calcolo probabilità (voce per voce).
- Mostra il rischio totale finale in percentuale, evidenziato con colore (verde/giallo/rosso in base alla soglia).
- Pulsante **"Esegui test degradazione"** → simula il lancio casuale.
- **Se degradazione avviene:**
 - Rimuove un effetto casuale dalla lista effetti risultanti.
 - Il grado scende di 1 (es. G3 → G2, quindi anche gli slot si riducono — rimuovere anche il secondo effetto casuale se necessario per rispettare il nuovo numero di

slot).

- Mostra messaggio drammatico con dettaglio di cosa è stato perso.
- **Se degradazione non avviene:** messaggio positivo.

STEP 5 — Riepilogo e Conferma

- Mostra la reliquia risultante completa: nome (generato automaticamente come "Reliquia Fusa [colore] G[grado]", modificabile dall'utente), proprietario, grado, grandezza, colore, effetti finali.
 - Mostra lo stadio catena calcolato.
 - Mostra se la reliquia è degradata.
 - Campo per inserire il **nome personalizzato** della reliquia risultante.
 - Campi **testimoni**: checkbox per selezionare i giocatori presenti come testimoni.
 - Pulsante "**Conferma Fusione**":
 - Aggiunge la nuova reliquia al database.
 - Marca le reliquie materiali come consumate (le rimuove dall'inventario o le segna come "usate" — suggerito: rimuoverle).
 - Aggiunge la voce al `registroFusioni`.
 - Mostra messaggio di successo.
 - Pulsante "**Annulla**" → torna allo step 1 senza salvare nulla.
-

SEZIONE 5 — TOKEN & RICOMPENSE

Gettone Cromatico (Art. 12)

- Assegnato ogni 1000 punti nel "Profondo della Notte".
- Form: seleziona giocatore, inserisci punti accumulati → il sistema calcola quanti gettoni spettano.
- Oppure: assegna manualmente un gettone cromatico a un giocatore.
- **Usa gettone cromatico:** seleziona una reliquia del proprietario → seleziona nuovo colore → conferma. Aggiorna il colore nel database e logga l'operazione in `registroToken`.

Gettone Creazione (Art. 13)

- Assegnato la prima volta che si raggiunge una nuova profondità.
- Form per assegnare manualmente a un giocatore (con campo "profondità raggiunta").
- **Usa gettone creazione:** form completo per creare una reliquia da zero (tutti i campi, effetti e step a scelta dall'elenco legit). Aggiunge la reliquia al database con nota "creata tramite Gettone Creazione".

Dashboard token

- Mostra per ogni giocatore: gettoni cromatici disponibili, gettoni creazione disponibili.
 - Storico utilizzi token.
-

SEZIONE 6 — REGISTRO FUSIONI

- Tabella cronologica di tutte le fusioni registrate.
 - Per ogni voce: data, proprietario risultato, reliquie materiali (con link/tooltip ai dettagli), reliquia risultante, se degradata, testimoni.
 - Filtri: per giocatore, per data, per esito (degradata sì/no).
 - Pulsante per esportare il registro come file CSV.
-

SEZIONE 7 — IMPOSTAZIONI

- Modifica nomi dei giocatori (con avviso che il cambio aggiorna tutti i riferimenti nel database).
 - Pulsante per resettare completamente il database (con conferma doppia).
 - Informazioni sulla versione del regolamento.
 - Link al regolamento (placeholder).
-

REGOLE DI BUSINESS — RIEPILOGO COMPLETO

Reliquie base

- Grado 1: 1 effetto slot. **NON possono essere usate come materiale di fusione.**
- Grado 2: 2 effetti slot.
- Grado 3: 3 effetti slot.

Grandezze

- Piccola, Media, Grande. Le reliquie fuse devono avere la stessa grandezza.

Colori

- Rosso, Blu, Giallo, Verde. Il colore non è un vincolo di compatibilità.

Tipologie

- `normale` : solo buff, pool da `effetti_normali_buff.json`
- `notte` : buff + debuff possibili, pool da `effetti_notte_buff.json` e `effetti_notte_debuff.json`
- **Fusione ibrida vietata:** normale + notte non si possono fondere.

Fusione — vincoli

1. Stessa tipologia.
2. Stesso grado (tutte G2 o tutte G3).
3. Stessa grandezza.
4. Nessuna reliquia di Grado 1.
5. Minimo 2 reliquie.

Fusione — risultato

- La reliquia risultante ha lo **stesso grado e grandezza** delle reliquie materiali.
- Il colore è determinato da sorteggio ponderato.
- Gli effetti sono sorteggiati dal pool unione degli effetti dei materiali.
- Numero di effetti risultanti = numero di slot del grado (es. G2 → 2 effetti).

Degradazione — calcolo probabilità

$$P = P_{base} + (n_{reliquie_fuse_materiali} * 10\%) + \text{somma per ogni materiale fuso co}$$

- $P_{base\ G2} = 10\%$, $P_{base\ G3} = 20\%$
- Cap massimo: non specificato nel regolamento, implementa senza cap (può arrivare al 100%)

Stadio catena — calcolo

```
se nessun materiale è fuso: stadioCartena = 0
altrimenti: stadioCartena = max(stadioCartena dei materiali fusi) + 1
```

Effetti e step

- Ogni effetto ha uno step corrente (1-based) e uno step massimo (definito nel JSON di configurazione).
- Upgrade: se un effetto viene estratto e già presente → step +1, purché non si superi stepMax e non sia già stato uppatato in questa fusione.
- Scarto: se estratto, già presente, e già a stepMax → scarta, ri-estrai.

STILE VISIVO

- **Palette:** sfondo nero/grigio scurissimo (#0d0d0d, #1a1a1a), testo bianco/grigio chiaro, accenti dorati/ambra (#c9a84c, #e8c96b) per titoli e pulsanti primari, accenti rosso scuro per warning/errori.
- **Font:** sans-serif moderno (es. Inter, Rajdhani, o simili caricati da Google Fonts).
- **Componenti:** card con bordo sottile ambra/dorato, badge colorati per i colori delle reliquie (pallino Rosso/Blu/Giallo/Verde), icone emoji o SVG minimali.
- **Animazioni:** sottili transizioni CSS per i sorteggi (effetto “spinning” testuale, nessuna animazione pesante).
- **Responsive:** funzionante su mobile e desktop.
- Il sito deve apparire **professionale e curato**, non amatoriale. Ogni sezione deve avere titoli chiari, istruzioni contestuali e feedback visivo per ogni azione.

FILE DA PRODURRE

Produci i seguenti file, tutti pronti per essere caricati in una repository GitHub e hostati su

GitHub Pages:

```
/
├─ index.html           ← SPA completa (HTML + CSS + JS inline)
├─ data/
│   ├─ effetti_normali_buff.json
│   ├─ effetti_notte_buff.json
│   └─ effetti_notte_debuff.json
└─ README.md           ← Istruzioni per l'uso (in italiano)
```

NOTE IMPLEMENTATIVE IMPORTANTI

1. **Tutti i sorteggi** (colore, effetti, degradazione) devono essere eseguiti con `Math.random()` in JavaScript. Mostrare sempre il risultato chiaramente prima di procedere al passo successivo.
 2. **Il sito non usa localStorage** per i dati principali. Tutto vive nel file JSON importato/esportato. (Puoi usare localStorage solo per preferenze UI secondarie come l'ultimo giocatore selezionato.)
 3. **IDs univoci:** usa `crypto.randomUUID()` per generare gli ID delle reliquie e delle fusioni.
 4. **Gestione errori:** se i file JSON degli effetti non sono trovati (404), mostrare un messaggio chiaro con istruzioni per crearli.
 5. **Esportazione JSON:** usa `Blob` e `URL.createObjectURL` per il download del file senza backend.
 6. **Caricamento JSON:** usa `FileReader` API per leggere il file importato.
 7. **Fetch effetti:** usa `fetch('./data/effetti_normali_buff.json')` ecc. al caricamento della pagina. Questo funziona su GitHub Pages se servito correttamente.
 8. Il wizard di fusione deve essere **stateful**: l'utente non deve poter saltare step, e tornare allo step precedente deve ripristinare lo stato corretto.
 9. Ogni azione distruttiva (elimina reliquia, reset database, conferma fusione) deve chiedere **conferma esplicita**.
 10. Il codice deve essere **ben commentato** in italiano, con sezioni chiaramente identificate.
-

DATI PLACEHOLDER — EFFETTI (da usare nei JSON)

`effetti_normali_buff.json` (almeno 10 effetti)

Esempi: Aumento Attacco (3 step), Aumento Difesa (3 step), Rigenerazione HP (2 step), Velocità Movimento (3 step), Resistenza Magia (3 step), Aumento Stamina (2 step), Riduzione Danni Fuoco (3 step), Potenziamento Critico (3 step), Aumento Range Attacco (2 step), Recupero FP (2 step).

`effetti_notte_buff.json` (almeno 10 effetti)

Esempi simili ma con valori più elevati nei nomi (es. "Attacco Notturmo +15%", "Potenziamento Oscuro", ecc.).

`effetti_notte_debuff.json` (almeno 8 effetti)

Esempi: Riduzione HP Massimi (3 step), Riduzione Difesa (2 step), Consumo FP Aumentato (2 step), Perdita Stamina Rapida (3 step), Vulnerabilità al Fuoco (2 step), Penalità Movimento (2 step), Riduzione Visuale (1 step), Maledizione Temporanea (3 step).

COMPORTAMENTO ATTESO IN PRODUZIONE

Un tipico flusso d'uso:

1. Un giocatore apre il sito, carica `reliquie.json`.
2. Va in "Fusione - Reliquie Normali", seleziona 2 reliquie G2 Piccole.
3. Estrae il colore (sorteggio ponderato simulato).
4. Estrae gli effetti uno per uno, gestendo eventuali duplicati.
5. Il sistema calcola la probabilità di degradazione e la mostra.
6. Clicca "Esegui test degradazione" → sorteggio casuale.
7. Inserisce il nome della reliquia risultante, seleziona i testimoni, conferma.
8. Esporta il file aggiornato e lo condivide con gli altri giocatori.

Fine del prompt. Realizza tutto il codice necessario per questo progetto, completo e funzionante.